DATA STRUCTURES & ALGORITHMS

Comprehensive Question Bank & Extended Analytical Solutions

Course Code: CS-201
Course Outcome: CO-1

Q.1: Define and explain: (i) Data, (ii) Data Structure, (iii) Flowchart

6 Marks

(i) Data & Information Hierarchy:

Data: Raw, unorganized, and atomic facts, figures, symbols, or observations (e.g., numbers, strings, readings) that lack context. Data alone cannot guide decision-making.

Information: Data that has been processed, structured, organized, or contextualized to provide meaning and utility (e.g., calculating the average score of a student class).

Key Distinction: `Information = Data + Context + Processing`.

(ii) Data Structure:

A Data Structure is a specialized organization format for storing, managing, and retrieving data efficiently in computer memory. It defines:

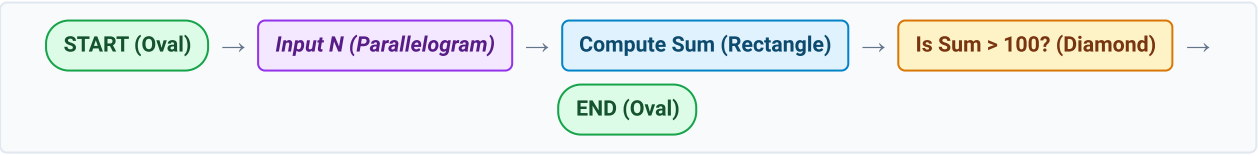
- **Logical Organization:** How elements relate to each other (e.g., linear sequence vs. hierarchical tree).
- **Storage Layout:** How values reside in memory (contiguous blocks vs. node pointers).
- **Operations Set:** Allowed manipulations such as Insertion, Deletion, Traversal, Searching, Sorting, and Merging.

(iii) Flowchart & Standard Graphical Notation:

A Flowchart is a standardized diagrammatic representation of an algorithm illustrating the logic and execution control flow.

Symbol Name	Geometric Shape	Algorithmic Purpose
Terminal	Oval / Rounded Rectangle	Marks the absolute Start or End of an algorithm.
Input / Output	Parallelogram	Denotes reading input data or printing/writing output results.
Process	Rectangle	Represents computations, assignments, or memory operations.
Decision	Diamond	Evaluates conditional branching expressions (True/False or Yes/No).
Flow Line	Arrowed Line	Indicates the exact sequence of execution.

Complete Flowchart Execution Pipeline:



Exam Tip: In theoretical exams, always draw all standard flowchart symbols with precise label titles and arrows indicating exact execution direction.

Q.2: Define Algorithm and explain its 6 essential characteristics

6 Marks

Formal Definition: An algorithm is an explicit, step-by-step, finite sequence of unambiguous instructions designed to solve a specific computational problem or produce a desired output from given inputs.

6 Mandatory Characteristics of a Valid Algorithm:

1. **Input:** An algorithm must accept *zero or more* well-defined external quantities.
2. **Output:** It must produce at least *one* output quantity that bears a mathematically sound relationship to the inputs.
3. **Definiteness (Unambiguity):** Every instruction must be explicit, crystal clear, and completely free of double interpretation or ambiguity.
4. **Finiteness:** The algorithm must unconditionally terminate after a finite number of steps for all valid input configurations. Infinite loops violate algorithm validity.
5. **Effectiveness (Feasibility):** Every operation must be sufficiently basic so that it can be carried out in practice using pencil and paper in finite time.
6. **Generality & Correctness:** The algorithm must correctly solve any problem instance belonging to the specified domain, rather than just a single hardcoded test case.

Standard Algorithmic Representation Example (Euclidean Algorithm for GCD):

```
Algorithm GCD(a, b):
// Input: Two non-negative integers a and b
// Output: Greatest Common Divisor of a and b
1. While b ≠ 0 do:
2.     temp ← b
3.     b ← a mod b
4.     a ← temp
5. Return a
```

Q.3: Explain Divide and Conquer strategy with an example and analyze time complexity

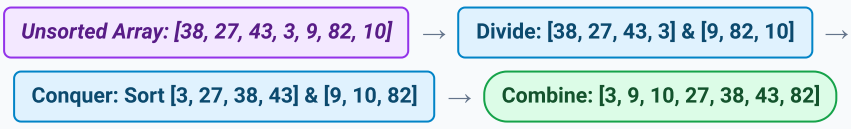
6 Marks

The **Divide and Conquer** design paradigm solves complex problems recursively by breaking them down into simpler, smaller sub-problems of the same type until they become trivial enough to solve directly.

The Three Algorithmic Steps:

- **1. Divide:** Partition the original problem of size *n* into *a* smaller sub-problems, each of size *n/b*.
- **2. Conquer:** Solve each sub-problem recursively. If sub-problem sizes are smaller than a threshold, solve them directly (base case).
- **3. Combine:** Merge the independent solutions of the sub-problems to construct the overall global solution.

Concrete Example — Merge Sort Algorithm Breakdown:



Mathematical Time Complexity Derivation (Master's Theorem):

The recurrence relation for Merge Sort is expressed as:

`T(n) = 2 T(n/2) + O(n)` (where $a = 2$ subproblems, $b = 2$ size division, and $f(n) = O(n)$ merge cost).

Using Master's Theorem: `n^(log_b a) = n^(log₂ 2) = n¹ = n`.

Since `f(n) = Θ(n^(log_b a)) = Θ(n)`, Case 2 applies:

`T(n) = Θ(n^(log_b a) · log n) =` **Θ(n log n)** (Holds across Best, Average, and Worst Cases).

Q.4: Explain Asymptotic Notations: Big O, Big Omega (Ω), and Big Theta (Θ)

6 Marks

Asymptotic notations provide a mathematical framework for expressing and comparing the asymptotic runtime and memory growth of algorithms as input size n approaches infinity.

Notation	Formal Mathematical Definition	Graphical Meaning	Practical Utility
Big O (O)	$0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$, $c > 0$	$f(n)$ is bounded from **above** by $c \cdot g(n)$.	Represents the **Worst-Case** guarantee / maximum runtime ceiling.
Big Omega (Ω)	$0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0$, $c > 0$	$f(n)$ is bounded from **below** by $c \cdot g(n)$.	Represents the **Best-Case** baseline / theoretical minimum runtime floor.
Big Theta (Θ)	$c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for $n \ge n_0$, $c_1, c_2 > 0$	$f(n)$ is tightly sandwiched between $c_1 \cdot g(n)$ & $c_2 \cdot g(n)$.	Represents the **Exact / Tight Bound** on growth rate.

Comparative Ranking of Common Growth Rates (Fastest to Slowest):

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

> **Exam Tip:** For mathematical proofs in exams, explicitly state the values of constants c and initial threshold n_0.

Q.5: Explain Static and Dynamic Data Structures with examples

6 Marks

Data structures are categorized into static or dynamic based on when memory allocation occurs and whether size flexibility exists during runtime execution.

Comparison Feature	Static Data Structure	Dynamic Data Structure
Memory Allocation Time	Compile-time allocation on Stack memory.	Run-time allocation on Heap memory.
Size Flexibility	Fixed capacity; cannot expand or shrink.	Dynamic capacity; grows/shrinks as needed.
Memory Efficiency	High risk of wastage (underflow) or overflow.	Optimal utilization; memory freed dynamically.
Access Speed & Layout	Contiguous memory; direct index access `O(1)`.	Non-contiguous pointer nodes; sequence traversal required.
Allocation Methods	Static array declaration: `int arr[100];`	Dynamic allocation: `malloc()`, `calloc()`, `free()` / `new`, `delete`.
Representative Examples	Fixed Arrays, Static Structures.	Singly/Doubly Linked Lists, Dynamic Trees, Dynamic Heaps.

Q.6: Differentiate between Linear and Non-Linear Data Structures

6 Marks

Attribute	Linear Data Structure	Non-Linear Data Structure
Element Arrangement	Elements are connected sequentially in a 1D linear sequence.	Elements are arranged hierarchically or interconnectedly.
Structural Levels	Single level of data representation.	Multiple hierarchical or non-hierarchical levels.
Element Traversal	Single pass linear traversal: `O(n)`.	Requires multi-path recursion or search (DFS/BFS traversal).
Neighbor Connectivity	Every element has 1 predecessor and 1 successor.	An element can connect to multiple child nodes or vertices.
Primary Applications	Task scheduling, stack operations, linear buffers.	Hierarchical file systems, network routing, AI game trees.
Examples	Arrays, Stacks, Queues, Linked Lists.	Binary Search Trees, AVL Trees, Heaps, Graphs.

Q.7: Explain Greedy Strategy with a suitable example and comment on its time complexity

6 Marks

The **Greedy Strategy** solves optimization problems by taking the locally optimal choice at each decision step, aiming to find a global optimum without revisiting previous choices.

Core Components of a Greedy Algorithm:

- **Candidate Set:** A pool of candidates from which a solution is constructed.
- **Selection Function:** Selects the best candidate to add to the partial solution.
- **Feasibility Function:** Determines if a candidate can contribute to a valid solution.
- **Objective Function:** Assigns a value to a solution or partial solution.

Classic Example — Fractional Knapsack Problem:

Given n items with values v_i and weights w_i, and a knapsack capacity W, maximize total value by taking fractions of items.

1. Calculate value-to-weight ratio $r_i = v_i / w_i$ for every item.
2. Sort all items in descending order of ratio r_i.
3. Greedily take full items while capacity allows; take a fraction of the next item when capacity runs out.

Time Complexity Breakdown:

- Ratio computation for n items: **O(n)**
- Sorting items by ratio: **O(n log n)**
- Knapsack selection loop: **O(n)**

Total Time Complexity: Dominated by the sorting phase = **O(n log n)**.

Q.8: Define and explain Abstract Data Type (ADT)

6 Marks

An **Abstract Data Type (ADT)** is a high-level mathematical specification of a data structure that defines *what* operations can be performed on the data without specifying *how* those operations are implemented in code.

Key Design Principles of ADTs:

- **Encapsulation & Abstraction:** Hides underlying data storage mechanisms (e.g., array vs. linked list) behind a clean public interface.
- **Modularity:** Allows implementations to be changed or optimized without altering client code that uses the ADT interface.

Formal Interface vs. Implementation Example (Queue ADT):

```
// Abstract Interface Specification
ADT Queue {
    Data: A linear ordered sequence of elements with FIFO access.
    Operations:
        enqueue(x) : Insert element x at the rear.
        dequeue()  : Remove and return element from the front.
        peek()     : Return front element without removing.
        isEmpty()  : Check if queue contains zero elements.
}
```

Q.9: State Analysis of Programming Constructs: Linear, Quadratic, Cubic, Logarithmic

6 Marks

Program runtime complexity is evaluated by analyzing loop bounds, nesting depth, and execution frequency of instructions.

Complexity Class	Loop Structure Pattern	Code Archetype	Common Example
Logarithmic O(log n)	Loop control variable multiplies or divides by constant $k > 1$.	`for(int i=1; i<n; i*=2)`	Binary Search, Binary Heap operations.
Linear O(n)	Single loop executing n iterations with constant increment step.	`for(int i=0; i<n; i++)`	Linear Search, Array Summation.
Quadratic O(n²)	Two nested loops, both dependent on input size n.	`for(i=0;i<n;i++)` `for(j=0;j<n;j++)`	Bubble Sort, Selection Sort, Matrix Addition.
Cubic O(n³)	Three nested loops dependent on input size n.	`for(i=0;i<n;i++)` `for(j=0;j<n;j++)` `for(k=0;k<n;k++)`	Standard Matrix Multiplication (n×n).

Q.10: Define and explain: (a) Linear DS, (b) Non-linear DS, (c) Time complexity, (d) Space complexity

6 Marks

(a) Linear Data Structure: Data structure where elements are sequentially organized in a single sequence. Access is strictly linear (e.g., Stack, Queue, Arrays).

(b) Non-Linear Data Structure: Data structure where elements are connected hierarchically or in complex networks, forming multi-level relationships (e.g., Trees, Graphs).

(c) Time Complexity: Quantification of total computational time required by an algorithm to execute to completion, expressed as a function of input size n.

(d) Space Complexity: Total memory consumed by an algorithm during execution, consisting of `Fixed Space + Variable Auxiliary Space + Call Stack Space`.

Q.11: Analyze time complexity of given code snippets using step count method

6 Marks

Snippet (i): Prime Number Check Analysis

```
bool Prime(int n) {
    for (int i = 2; i < n; ++i) {  // Line 1: i=2 to n
        if (n % i == 0)            // Line 2: Modulo test
            return false;          // Line 3: Early exit
    }
    return true;                   // Line 4: Final exit
}
```

Line No.	Operation	Unit Cost	Worst-Case Execution Frequency	Total Step Cost
Line 1	Loop control initialization & test	1	n - 1 times (i = 2 to i = n)	n - 1
Line 2	Modulo condition check `(n % i == 0)`	1	n - 2 times (i = 2 to i = n - 1)	n - 2
Line 4	Return statement `return true;`	1	1 time	1

Total Step Count T(n): $(n - 1) + (n - 2) + 1 = 2n - 2$.
Worst-Case Asymptotic Time Complexity: O(n).

Snippet (ii): Nested Logarithmic Loops Analysis

```
void f() {
    int ans = 0;
    for (int i = n; i >= 1; i /= 2) {      // Outer Loop: divides i by 2
        for (int j = 1; j <= m; j *= 2) {  // Inner Loop: multiplies j by 2
            ans += (i * j);
        }
    }
    print(ans);
}
```

• Outer loop sequence for *i*: *n, n/2, n/4, ..., 1*. Total iterations: $\lfloor \log_2 n \rfloor + 1$.
• Inner loop sequence for *j*: *1, 2, 4, ..., m*. Total iterations per outer step: $\lfloor \log_2 m \rfloor + 1$.
• Total inner execution steps: $(\log_2 n + 1) \times (\log_2 m + 1)$.
Overall Asymptotic Time Complexity: O(log n · log m).

Q.12: Write short notes on: 1. ADT, 2. Divide & conquer, 3. Greedy, 4. Array, 5. Space complexity, 6. Time complexity

6 Marks

1. Abstract Data Type (ADT): Theoretical interface defining data objects and allowable operations, completely abstracting away low-level implementation details.

2. Divide and Conquer: Recursive strategy that splits a problem into independent sub-problems, solves them recursively, and combines results (e.g., Merge Sort).

3. Greedy Strategies: Paradigm making locally optimal choices at each step without backtracking to build a global solution (e.g., Prim's MST, Knapsack).

4. Array: Linear data structure allocating contiguous memory locations for storing elements of identical data types, offering `O(1)` index lookup.

5. Space Complexity: Memory resources required by an algorithm, comprising fixed code space, variable space, and recursive call stack space.

6. Time Complexity: Asymptotic metric describing how the execution time of an algorithm scales as input size *n* grows.

Q.13: What is meant by algorithm? Explain the characteristics of an algorithm

6 Marks

An algorithm is an explicit, ordered sequence of computational steps that transforms valid inputs into a desired output within finite execution steps.

Key Characteristics:

- **Finiteness:** Must terminate after a finite number of steps.
- **Definiteness:** Steps must be unambiguous and uniquely defined.
- **Input:** Accepts 0 or more valid inputs.
- **Output:** Produces at least 1 valid result.
- **Effectiveness:** Operations must be simple enough to be performed directly.

Q.14: Differentiate between linear and non-linear data structures with examples

6 Marks

Linear data structures arrange elements sequentially in 1D space (Arrays, Stacks, Queues, Linked Lists), supporting single-pass `O(n)` traversal. Non-linear structures arrange elements hierarchically or across interconnected network graphs (Trees, Graphs), requiring recursive or queue-based multi-path traversal algorithms (DFS/BFS).

Q.15: What is time complexity? Why are asymptotic notations important? Find time complexity of code using step count

6 Marks

Importance of Asymptotic Notation: Enables hardware-independent analysis of algorithm efficiency, allowing comparison of algorithms based on scaling behavior as input size *n* approaches infinity.

Step Count Derivation:

```
int sum(int arr[], int n) {
    int s = 0;                    // Step 1: 1 time
    for (int i = 0; i < n; i++) { // Step 2: n + 1 comparisons
        s += arr[i];              // Step 3: n additions
    }
    return s;                     // Step 4: 1 time
}
```

Total Step Count: `T(n) = 1 + (n + 1) + n + 1 = 2n + 3`.
Asymptotic Complexity: O(n).

Q.16: Explain algorithmic efficiency and performance analysis (Space vs Time tradeoff) — 6 Marks

Performance Analysis: Evaluates algorithm quality based on time complexity (execution speed) and space complexity (memory usage).

Space-Time Tradeoff Principles:

- **Time Optimization (Using Space):** Storing intermediate calculations in auxiliary tables (e.g., Dynamic Programming memoization tables, Hash Tables) enables fast `O(1)` lookups at the cost of extra memory.
- **Space Optimization (Using Time):** Recomputing values in-place (e.g., In-Place Bubble Sort) saves memory (`O(1)` space) at the cost of higher execution time (`O(n²)`).

Q.17: Explain Step Count method in detail with a step-by-step tabular example — 6 Marks

The **Step Count Method** counts the total number of basic operations executed by an algorithm as a function of input parameters.

Tabular Analysis — Matrix Addition (n × n):

```
void addMatrix(int A[][MAX], int B[][MAX], int C[][MAX], int n) {
    for (int i = 0; i < n; i++) {          // Line 1
        for (int j = 0; j < n; j++) {      // Line 2
            C[i][j] = A[i][j] + B[i][j];   // Line 3
        }
    }
}
```

Line No.	Statement	Cost	Frequency	Total Steps
Line 1	Outer loop condition	1	n + 1	n + 1
Line 2	Inner loop condition	1	n × (n + 1)	$n^2 + n$
Line 3	Matrix addition assignment	1	n × n	n^2

Total Step Function: `T(n) = (n + 1) + (n² + n) + n² = 2n² + 2n + 1` steps.
Asymptotic Complexity: O(n²).

Q.18: Explain Best Case, Worst Case, and Average Case analysis with suitable examples — 6 Marks

Algorithm performance varies depending on input configuration and data ordering:

- **Best-Case Analysis:** Minimum execution steps across all inputs of size *n* (e.g., Linear Search finding element at index 0 yields **Ω(1)**).
- **Worst-Case Analysis:** Maximum execution steps across all inputs of size *n*, defining an absolute performance upper bound (e.g., Linear Search on an absent element yields **O(n)**).
- **Average-Case Analysis:** Expected execution steps averaged over all possible input permutations under an assumed uniform distribution (e.g., Linear Search average steps = `(n + 1) / 2`, giving **Θ(n)**).

> **Exam Tip:** System architects rely primarily on **Worst-Case Analysis** because it provides a reliable, guaranteed upper bound on execution performance.